

Task 3 — Submission Report

From Notebooks to Production

Student	Ziad Suleyman
Course	MLOPS Training — 2026/2027
Trainer	Trainer Qafza
Problem	Predict late vs on-time delivery
Deliverable	FastAPI inference service, v1.2.0
Repository	github.com/ziadSuleyman/MLOPS → <code>mlops-task3/</code>
Commit	<code>a5b49cb</code> · CI green
Model version	<code>f9fcc48ef9ef</code> (hash of the artifacts)
Tests	173 passed, 0 skipped
Report date	24 September 2026

Every console output in this report was captured on 24 September 2026 from a fresh `git clone` of the repository, started with one command on empty Docker volumes.

0. Definition of done — self-check against the task sheet

You should have

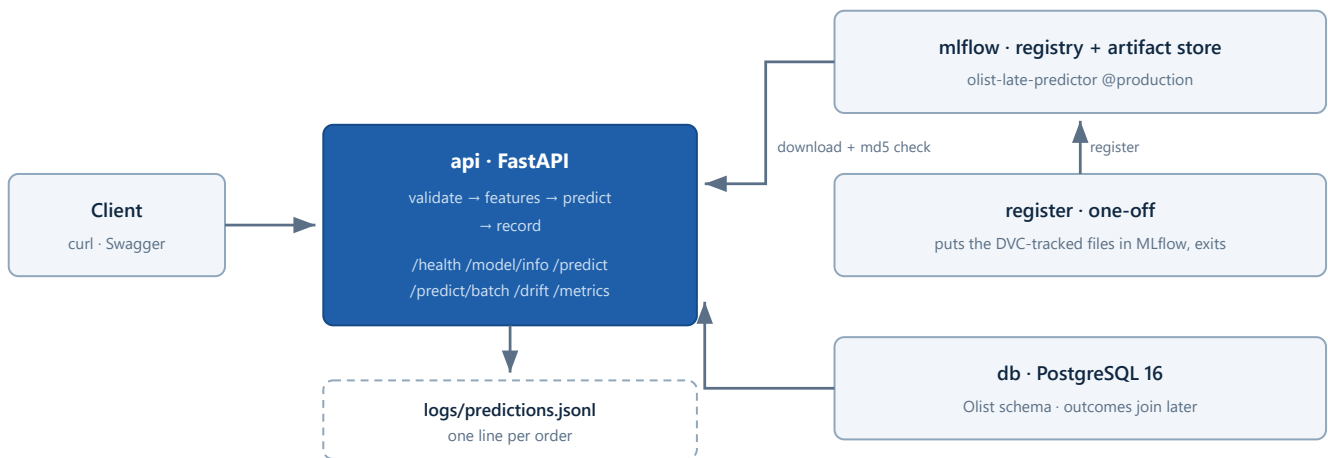
Criterion (from the task PDF)	Status	Evidence
1. A structured repository with config files, requirements, and a README that works from zero	Done	§1 folder layout · §4 the README's Quick Start run on a fresh clone
2. An inference pipeline in Python modules that loads the saved fitted objects and the registered model	Done	§2 modules · §7 the API loads <code>olist-late-predictor@production</code> from MLflow
3. Data versioning with DVC, data validation with Great Expectations, tracking with MLflow	Done	§6 validation · §7 DVC pointers + MLflow run, registry, alias and stage
4. Unit tests and integration tests that pass with one command	Done	§8 <code>pytest</code> → 173 passed, 0 skipped
5. A FastAPI service with health, model info, and predict routes	Done	§5 six routes, schema-validated, Swagger docs
6. Docker Compose bringing up the database, the API, and the artifact storage together	Done	§4 <code>db</code> + <code>mlflow</code> (artifact store) + <code>register</code> + <code>api</code>
7. A CI/CD pipeline that runs the tests and builds the image	Done	§9 lint → tests → build & push to ghcr.io
8. Logging and monitoring in place, with prediction logs stored	Done	§10 service log, JSONL prediction log, Prometheus metrics, PSI drift

You should be able to

Criterion (from the task PDF)	Status	Evidence
1. Clone the repo on a clean machine and start everything with one command	Done	§4 <code>docker compose up --build</code> : four services, 2 min 5 s
2. Send an order to the predict route and get back a prediction, a probability, and a model version	Done	§5 <code>POST /predict</code> on a real test-split order
3. Show that the pipeline output matches your notebook output on the same input	Done	§3 max score difference 2.2×10^{-16} over 18,603 orders; ROC-AUC $0.7107 = 0.7107$
4. Break something on purpose — bad data, a failing test — and show the system catches it	Done	§6 eight bad inputs · §8 a code bug · §9 a failing CI run · §7 registry down
5. Explain every folder, every config value, and every tool you used, and why it is there	Done	§1 folders · §11 config values · §12 tools

1. The repository

The whole training track lives in one git repository; Task 3 is the folder `mlops-task3/`. Its layout is the one the task sheet asks for, with no hardcoded paths or parameters in the code.



Folder / file	What is in it, and why it is there
<code>app/</code>	The FastAPI service: <code>main.py</code> (routes, start-up checks, error handling) and <code>schemas.py</code> (Pydantic request/response models). Kept apart from <code>src/</code> so the pipeline can run without a web server.
<code>src/</code>	The inference pipeline, one concern per module (§2). Everything the API and the scripts share.
<code>config/settings.yaml</code>	Every path, parameter and threshold (132 lines). Secrets are not here; they come from <code>.env</code> (§11).
<code>models/</code>	The four serving artifacts and their four <code>.dvc</code> pointer files. The same bytes are registered in MLflow.
<code>data/</code>	Placeholder for input data. The service reads no data files: orders arrive in the request, outcomes come from the database.
<code>notebooks/</code>	The six Task 2 notebooks, byte-for-byte reference copies. Not in the image, not linted.
<code>tests/</code>	173 pytest tests in 14 modules, plus <code>fixtures/parity_orders.parquet</code> (314 real orders with the notebook's features).
<code>scripts/</code>	Offline operations: build the serving reference, build the parity fixture, register the model, compare local vs registry, analyse the prediction log, score it against real deliveries.
<code>requirements/</code>	<code>base.txt</code> = runtime (in the image), <code>dev.txt</code> = <code>-r base.txt</code> + <code>pytest</code> , <code>ruff</code> , <code>pre-commit</code> , <code>DVC</code> , <code>Jupyter</code> . Every version pinned.
<code>logs/</code>	Runtime output: <code>service.log</code> and <code>predictions.jsonl</code> . Gitignored; bind-mounted from the container.
<code>docs/</code>	<code>ARCHITECTURE.md</code> (modules, data flow, error table) and <code>DEPLOYMENT.md</code> (deploy, update the model, post-deploy checklist).

Folder / file	What is in it, and why it is there
Dockerfile , docker-compose.yml , .env.example	The image, the four-service stack, and the template for secrets (§4).
.dvc/ , .pre-commit-config.yaml , pyproject.toml	DVC remote config; the hooks run before every commit; ruff and pytest settings.
../.github/workflows/task3-ci.yml	The CI pipeline. It sits at the repository root because GitHub reads workflows only from there.

Pinned requirements

The runtime is pinned to the environment the Task 2 artifacts were pickled in: Python 3.13, scikit-learn 1.9.0, pandas 3.0.5, numpy 2.5.1. The MLflow client is pinned to the server image (3.16.1). `src/artifacts.load_pickle` refuses to load the model under any other scikit-learn version rather than log a warning and serve anyway.

EVIDENCE 1 — requirements/base.txt (runtime, excerpt)

```
fastapi==0.141.1      pandas==3.0.5      scikit-learn==1.9.0  pyyaml==6.0.3
uvicorn[standard]==0.52.4 numpy==2.5.1      scipy==1.18.0        python-dotenv==1.2.3
pydantic==2.13.5      pyarrow==25.0.1    joblib==1.5.3         mlflow==3.16.1
pydantic-settings==2.15.0 sqlalchemy==2.0.51 great-expectations==1.23.0
                        psycpg[binary]==3.3.4 prometheus-client==0.26.0
# dev.txt: -r base.txt + pytest==9.1.1 pytest-cov==7.1.0 httpx==0.28.1 ruff==0.16.7
#           pre-commit==4.2.0 dvc==3.67.1 jupyterlab==4.6.2 ipykernel==7.3.0
```

2. From notebooks to Python modules

Only the inference half of the notebooks was ported; training stays in Notebook 6. Each module does one job, and the fitted objects are loaded, never fitted.

Module	Ported from	Responsibility
src/config.py	—	Loads <code>settings.yaml</code> + <code>.env</code> into typed objects; invalid settings stop the start-up
src/logger.py	—	Rotating file handler (10 MB × 5) + console handler
src/validation.py	NB 1–2 (data facts)	Per-order rules: fatal problems vs. rule problems
src/ge_validation.py	NB 4 (ranges)	Great Expectations suite, 70 expectations, for batches
src/preprocessing.py	NB 5 §2 <code>build_derived</code>	Pure row-level transforms: haversine distance, promised days, ratios, flags
src/features.py	NB 5 transformers	Applies the saved target encoders, one-hot encoder and imputer → exactly 31 features
src/predict.py	NB 6	Loads the model, scales, scores, calibrates, applies the alert threshold
src/reference.py , src/alerting.py	NB 6 (alert budget)	Calibration, risk percentile, PSI bins; the 5% alert budget as a rolling threshold

Module	Ported from	Responsibility
<code>src/pipeline.py</code>	—	Orchestrates <code>validate</code> → <code>features</code> → <code>predict</code> → <code>record</code> , for one order or a batch
<code>src/monitoring.py</code>	—	Prometheus metrics, PSI drift window, JSONL prediction log
<code>src/evaluation.py</code>	NB 2 (label)	Scores logged predictions against real deliveries with the calendar-day label
<code>src/artifacts.py</code> , <code>src/registry.py</code>	—	Model version = hash of the artifacts; DVC pointer check; MLflow register and load

Fitted objects are loaded, never fitted

The service loads four artifacts. The first two are Task 2's files, unchanged:

- `05_transformers.joblib`: `kept_categories` , `target_encoders` , `onehot_payment` , `imputer` , `feature_columns` , `scaler` , `metadata`
- `06_model.joblib`: `LogisticRegression(C=0.3, class_weight="balanced")` , 31 features
- `05_feature_list.json`: the 31-feature contract
- `07_serving_reference.json`: Platt calibration and the validation-score percentiles. Built offline from the **validation** split by `scripts/build_serving_reference.py` . It records the model's md5, and the service refuses it for any other model.

The config key `inference.refit` is `false` , and the service refuses to start if it is set to `true` . The start-up log lists exactly what was loaded:

EVIDENCE 2 — logs/service.log at start-up (clean stack)

```
[2026-09-24 16:56:34] [INFO] [olist] Starting Olist Late Delivery Predictor v1.2.0
[2026-09-24 16:56:36] [INFO] [olist] Loading transformers from /tmp/olist-registry-wc0oqcd0/serving/05_transformers.joblib
[2026-09-24 16:56:37] [INFO] [olist] Transformers loaded: ['kept_categories', 'target_encoders',
'onehot_payment',
                                'imputer', 'feature_columns', 'scaler', 'metadata'] | 31 features
[2026-09-24 16:56:37] [INFO] [olist] Loading model from /tmp/olist-registry-wc0oqcd0/serving/06_model.joblib
[2026-09-24 16:56:37] [INFO] [olist] Model loaded: LogisticRegression | 31 features |
requires_scaling=True
[2026-09-24 16:56:37] [INFO] [olist] Serving reference loaded: built on validation (13547 orders, late rate
5.53%)
[2026-09-24 16:56:38] [INFO] [olist] Serving model f9fcc48ef9ef
```

The paths are under `/tmp/olist-registry-.../serving/`: the files were downloaded from the MLflow registry, not read from the image's `models/` folder (§7).

3. The pipeline reproduces the notebooks

This was checked in two ways, both on real test-split orders:

1. **Features.** Raw orders go through `src/` . The 31 features that come out are compared with the values Notebook 5 wrote to `05_features_test.parquet` .
2. **Scores.** The model is applied to the service's features and to the notebook's features, and the scores are compared. On the full test split the two ROC-AUC values must match.

EVIDENCE 3 — service vs. notebooks, measured inside the service image

```
Fixture: 314 real test-split orders x 31 features
max |service feature - notebook feature| = 4.547e-13
max |service score - notebook score| = 0.000e+00
Full test split: 18603 orders
max |service score - notebook score| = 2.220e-16
ROC-AUC service 0.7107 notebook 6 0.7107
PR-AUC service 0.0849 notebook 6 0.0849
```

The 314-order fixture includes the awkward cases on purpose: missing customer coordinates, a missing category, missing photos, east-coast longitudes, and coordinates outside Brazil. It is committed, so `tests/test_parity.py` runs these checks in CI on every push (§8). The largest feature difference, 4.5×10^{-13} , is floating-point rounding in the haversine formula.

Parity also holds through the whole running system. The 18,603 orders were sent to the live API, logged, and later scored against the database (§10.4). That run gave the same ROC-AUC, 0.7107.

4. One command on a clean machine

The repository was cloned from GitHub into an empty folder and started with the README's Quick Start. The compose project was given a new name (`-p task3clean`), so Docker created every volume empty. No state came from earlier runs: no database, no MLflow registry, no cached model.

EVIDENCE 4 — clone and start

```
$ git clone https://github.com/ziadSuleyman/MLOPS.git && cd MLOPS/mlops-task3
$ git log -1 --format="%h %s"
a5b49cb Task 3: CI pushes the image to ghcr.io after lint and tests pass
$ cp .env.example .env
$ API_PORT=8010 docker compose -p task3clean up --build -d          # 8000 is taken on this machine
Volume task3clean_mlflow_data Created
Container task3clean-mlflow-1 Healthy
Container task3clean-db-1 Healthy
Container task3clean-registry-1 Exited
Container task3clean-api-1 Started
real    2m5.679s

SERVICE    STATUS                PORTS
api          Up 2 seconds          0.0.0.0:8010->8000/tcp
db           Up About a minute (healthy) 0.0.0.0:5433->5432/tcp
mlflow       Up About a minute (healthy) 0.0.0.0:5000->5000/tcp
register      Exited (0)
```

The start-up order is enforced by `depends_on` conditions:

1. `db` and `mlflow` start and pass their health checks.
2. `register` puts the model in the empty registry and exits with code 0.
3. `api` starts only after that exit, and loads the model back from the registry.

EVIDENCE 5 — the one-off register step, on an empty MLflow

```
Experiment with name 'olist-late-predictor' does not exist. Creating a new experiment.
Successfully registered model 'olist-late-predictor'.
Created version '1' of model 'olist-late-predictor'.
[INFO] [olist] Registered f9fcc48ef9ef as olist-late-predictor version 1 (run
a5ae01770dde4628b4ba61d8c2e13509)
[MLflow] Alias 'production' and stage 'Production' -> version 1
```

The image

Base	python:3.13-slim
Size	302 MB compressed. Most of it is the pinned ML stack: <code>site-packages</code> holds 867 MB unpacked (MLflow, Great Expectations, scikit-learn, pandas).
Contents of <code>/app</code>	<code>app config logs models requirements scripts src</code> : no notebooks, no tests, no dev tools
User	<code>appuser</code> (non-root), which owns <code>logs/</code>
Secrets	None in the image or the repository. <code>DB_*</code> and <code>MLFLOW_TRACKING_URI</code> come from the environment; <code>.env</code> is gitignored and <code>.env.example</code> is the template.

The `db` service uses its own empty volume with the Olist schema. The prediction service does not read it. The historical orders used to score predictions (§10.4) are in the Task 1 database container, which uses the same port, 5433.

5. The API

Route	Returns
GET <code>/health</code>	Status, model loaded, <code>model_version</code> , service version, and where the model came from
GET <code>/model/info</code>	Model type, 31 features, hyperparameters, validation and test metrics, md5 of every artifact, calibration, alert policy
POST <code>/predict</code>	One order → prediction, probability, score, risk percentile, alert threshold, model version, latency
POST <code>/predict/batch</code>	Up to 500 orders (<code>inference.batch_max_size</code>); each one is validated, predicted and logged
GET <code>/drift</code>	PSI of the last 1,000 scores against the validation split, alert ratio, mean probability
GET <code>/metrics</code>	Prometheus text format (§10.2)

EVIDENCE 6 — GET `/health`

```
{
  "status": "healthy",
  "model_loaded": true,
  "model_version": "f9fcc48ef9ef",
  "service_version": "1.2.0",
  "model_source": "olist-late-predictor@production
                  (version 1, run a5ae0177...)"
}
```

EVIDENCE 7 — GET `/model/info` (excerpt)

```
"model_type": "LogisticRegression",
"n_features": 31,
"hyperparameters": {"C": 0.3,
  "class_weight": "balanced", "max_iter": 3000},
"label_definition": "calendar-day rule",
"metrics": {
  "validation": {"ROC-AUC": 0.7739, "PR-AUC":
0.1709},
  "test":       {"ROC-AUC": 0.7107, "PR-AUC":
0.0849}},
"artifacts_md5": {"model": "f352d02d...", ...}
```

A prediction

The request below is order `8a9be36f...`, a real order from the test split. It is also the example body shown in the Swagger docs, so the docs example returns 200.

EVIDENCE 8a — POST /predict (request)

```
{
  "order_id": "8a9be36ffd78382f9ac518945e909636",
  "order_purchase_timestamp": "2018-06-01
04:19:11",
  "order_estimated_delivery_date": "2018-07-05",
  "shipping_limit_first": "2018-06-11 04:30:37",
  "customer_state": "MG", "main_seller_state":
"SP",
  "customer_lat": -19.5175, "customer_lng":
-42.6117,
  "seller_lat": -23.6524, "seller_lng": -46.7555,
  "freight_total": 18.43, "freight_max": 18.43,
  "items_price_total": 78.0, "items_price_max":
78.0,
  "payment_total": 96.43, "installments_max": 4,
  "n_payments": 1, "main_payment_type":
"credit_card",
  "n_items": 1, "n_products": 1, "n_sellers": 1,
  "n_categories": 1, "weight_g_total": 250,
  "photos_avg": 4, "main_category":
"relogios_presentes"
}
```

EVIDENCE 8b — response 200

```
{
  "order_id": "8a9be36ffd78382f9ac518945e909636",
  "prediction": "on_time",
  "probability": 0.0221,
  "score": 0.3197,
  "risk_percentile": 20.8,
  "alert_threshold": 0.7538,
  "model_version": "f9fcc48ef9ef",
  "latency_ms": 165.14,
  "validation_warnings": null
}
```

EVIDENCE 8c — same order, promise cut to 7 days, customer moved to São Luís (MA)

```
"prediction": "late",
"probability": 0.6835,
"score": 0.9664,
"risk_percentile": 99.8,
```

The response has three outputs:

- **probability** is calibrated. The model's raw **score** is not a probability: it was trained class-balanced, so it averages 0.49. Platt scaling, fitted on the validation split, maps the score to a probability. On the test split the Brier score fell from 0.2685 (raw) to 0.0379 (calibrated).
- **prediction** is **late** when the order is in the riskiest 5% of recent orders (the alert budget from Notebook 6). A 0.5 cut-off would flag 49.3% of test orders; the budget flagged 5.4% (\$10.3).
- **model_version** is not typed by hand. It is a hash of the four artifacts, so any change to them changes it.

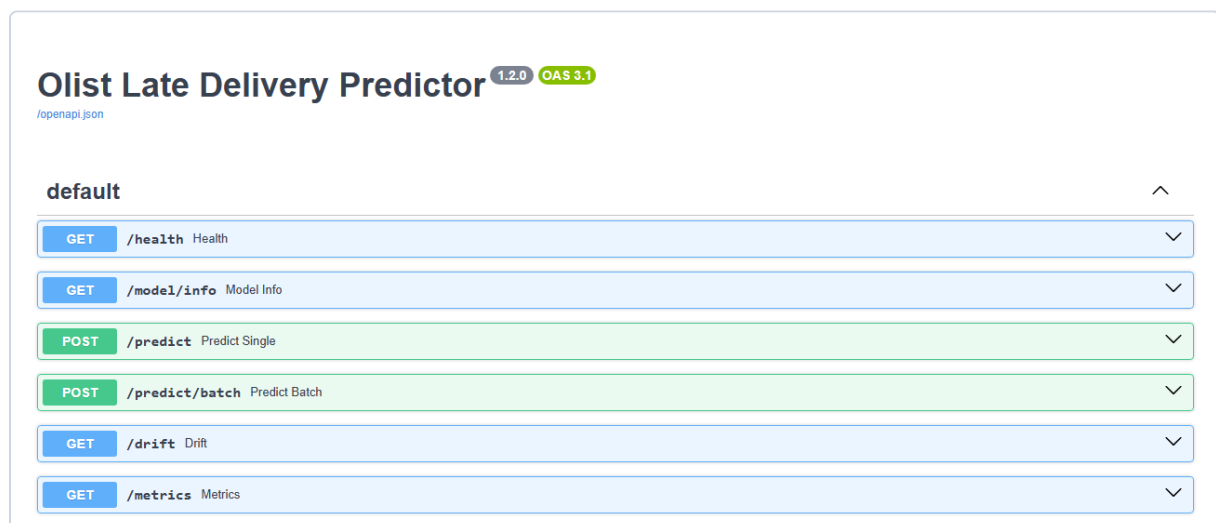


Figure 1 — the automatic docs at `/docs` on the clean stack: six routes and eight request/response schemas

6. Data validation and error handling

An order passes three checks before it reaches the model. Every rejection is a clear HTTP 422, and the service keeps running.

Layer	Runs on	Checks
Pydantic schema (<code>app/schemas.py</code>)	every request	Required fields, types, no NaN or Infinity, batch size ≤ 500
Per-order rules (<code>src/validation.py</code>)	every order	Dates parseable and promise after purchase; states among the 27 Brazilian states; payment type known; money and weight ≥ 0 ; counts ≥ 1 ; coordinates on Earth. None of the 96,470 real orders breaks a rule.
Great Expectations (<code>src/ge_validation.py</code>)	every batch	70 expectations: column existence (24), numeric ranges (13), coordinates within Brazil's bounding box (4), allowed categories (3), missing-rate limits (22), positive counts (4), no leakage columns (1)

The Great Expectations suite describes a population ("95% of freight values in the usual range", "at most 10% missing coordinates"), so it runs on batches rather than on single orders. One run takes about 0.9 s; one prediction takes about 40 ms.

Policy when validation fails (`validation.on_failure`):

- `reject` (the default): HTTP 422 with the list of problems.
- `flag`: the order is scored anyway, and the problems are attached to the response, the log and a Prometheus counter.
- Problems that make scoring impossible (missing or unreadable dates, non-numbers) always reject, whatever the policy.

Breaking it on purpose — bad data

Eight bad requests were sent to the clean stack. All eight were caught, and `/health` stayed `healthy` afterwards.

#	What was sent	Status	Service answer
1	<code>freight_total: -5.0</code>	422	<code>"freight_total must be >= 0, got -5.0"</code>
2	<code>customer_state: "XX"</code>	422	<code>"Unknown customer_state: 'XX'. Allowed: ['AC', 'AL', ... 'TO']"</code>
3	promise date before the purchase	422	<code>"order_estimated_delivery_date must be after order_purchase_timestamp"</code>
4	<code>order_purchase_timestamp</code> missing	422	<code>"type": "missing", "loc": ["body", "order_purchase_timestamp"]</code>
5	<code>freight_total: "eighteen"</code>	422	<code>"float_parsing" ... "unable to parse string as a number"</code>
6	malformed JSON <code>{not json</code>	422	<code>"type": "json_invalid", "msg": "JSON decode error"</code>
7	batch of 501 orders	422	<code>"List should have at most 500 items after validation, not 501"</code>

#	What was sent	Status	Service answer
8	batch of 40 orders, half with freight 4,800 (each legal alone)	422	"Batch validation failed" — Great Expectations (below)

EVIDENCE 9 — case 8: every order passes the per-order rules; the batch fails Great Expectations

```
HTTP 422
{
  "detail": "Batch validation failed",
  "errors": [{ "batch": [
    "batch check failed: expect_column_values_to_be_between on freight_total",
    "batch check failed: expect_column_values_to_be_between on freight_max" ] } ]
}
```

Each rejection is counted in `prediction_errors_total{error_type="validation"}` (§10.2). Model errors and unexpected errors return 500 with the label `internal`. The service also refuses to *start* in any of these cases:

- `refit: true`, or an unknown `on_failure` or `model.source` value
- registry files whose md5 differs from what was recorded for that version
- a scikit-learn version mismatch
- a serving reference built for another model
- a leakage column in the feature contract

7. Versioning: DVC and MLflow

7.1 DVC — the artifacts

The four serving artifacts are tracked by DVC, and each `.dvc` pointer records the file's md5. The remote, `localstore`, is a folder outside the repository. It has been pushed to and pulled from.

EVIDENCE 10 — DVC state

```
$ python -m dvc status
Data and pipelines are up to date.
$ python -m dvc status -c
Cache and remote 'localstore' are in sync.
$ cat models/06_model.joblib.dvc
outs:
- md5: f352d02dae6a49f48e499dfd54c85e0b
  size: 2280
  path: 06_model.joblib
```

The artifacts are small (1–8 KB), so they are also committed to git and copied into the image. That lets CI run the tests without access to the DVC remote. The DVC remote is a local folder. Copies outside this laptop are the image on `ghcr.io/ziadsuleyman/olist-late-predictor` and the MLflow artifact store volume.

7.2 MLflow — tracking and registry

`scripts/register_model.py` creates one run in the experiment `olist-late-predictor`. The run logs the parameters, the validation and test metrics, and the four serving files. The model is registered as `olist-late-predictor`, and the new version gets the alias `production` and the stage `Production`. MLflow deprecated stages in 2.9, so both are set, and the service reads the alias.

EVIDENCE 11 — the clean stack's registry, read back through the MLflow REST API

```
aliases: [{'alias': 'production', 'version': '1'}]
version: 1 stage: Production status: READY
version tags: model_version=f9fcc48ef9ef
               md5_model=f352d02dae6a49f48e499dfd54c85e0b   md5_transformers=6a175ab09db74cb40e8d2d3877e928a8
               md5_feature_list=c8a67aba56ef8826f1587c4159233b18
md5_reference=3e7e841f57893f37f0705fa3d0e9bb2e
params: C=0.3 class_weight=balanced max_iter=3000 n_features=31 requires_scaling=True
        label_definition=calendar-day rule (notebook 2)
metrics: validation_ROC-AUC=0.7739 validation_PR-AUC=0.1709 validation_recall_at_5pct=20.7
validation_lift=4.14
        test_ROC-AUC=0.7107      test_PR-AUC=0.0849      test_recall_at_5pct=15.0      test_lift=3.01
artifacts (serving/): 05_feature_list.json 1301 B   05_transformers.joblib 7779 B
                     06_model.joblib 2280 B      07_serving_reference.json 2842 B
```

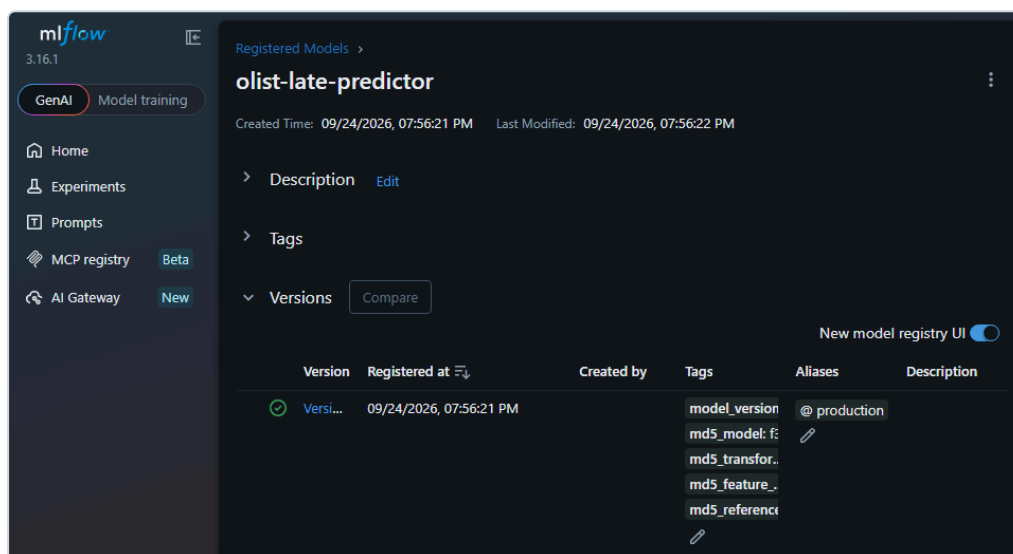


Figure 2 — the MLflow UI on the clean stack: version 1 carries the md5 tags and the alias @ production

7.3 One chain of hashes

At start-up the API looks up `olist-late-predictor@production` and downloads that version's `serving/` files. It checks each file's md5 against the version's tags **and** against the DVC pointers, and loads the files only if both match. The same md5 therefore appears in four places:

- the DVC pointer (`models/06_model.joblib.dvc`)
- the MLflow version tag (`md5_model`)
- `GET /model/info → artifacts_md5.model`
- the `model_version` hash reported in every response

Situation	What the API does
Registry reachable, files check out	Serves the registry version; <code>model_source</code> names it
Registry unreachable or not configured	Serves the same DVC-tracked bytes from <code>models/</code> , and says so
Registry serves bytes different from those recorded	Refuses to start (tested: <code>test_tampered_version_is_refused</code>)

EVIDENCE 12 — breaking it on purpose: MLflow stopped, API restarted

```
$ docker compose -p task3clean stop mlflow && docker compose -p task3clean restart api
[WARNING] [olist] Model source: models/ (registry olist-late-predictor@production unavailable:
    <urlopen error [Errno -5] No address associated with hostname>)
$ curl localhost:8010/health
{"status": "healthy", "model_loaded": true, "model_version": "f9fcc48ef9ef", "service_version": "1.2.0",
 "model_source": "models/ (registry olist-late-predictor@production unavailable: ...)"}

```

The same `model_version` was served from the fallback. After MLflow was started again, a restart returned `model_source` to `olist-late-predictor@production` (version 1, ...).

8. Testing

The whole suite runs with one command, `pytest`. It was run here inside the image the clean stack built, with the Task 2 artifacts mounted, so the optional full-test-split parity test ran as well.

EVIDENCE 13 — pytest inside olist-late-predictor (clean clone)

```
$ python -m pytest
...
===== 173 passed, 47 warnings in 131.10s (0:02:11) =====

```

The 47 warnings are scikit-learn's `X has feature names, but LogisticRegression was fitted without feature names`. The Task 2 model was fitted on a numpy array; the service passes the columns in the contract's order, and parity confirms the result.

Module	Tests	Kind	Covers
<code>test_preprocessing.py</code>	11	unit	haversine, derived columns, edge cases
<code>test_features.py</code>	15	unit / data	transformers applied, exactly 31 features, no NaN, no leakage, Infinity handled
<code>test_validation.py</code>	25	data	every rule, fatal vs. problem, all 96,470 real orders pass
<code>test_ge_validation.py</code>	15	data	GE ranges, sets, null rates, leakage, a real batch, east-coast longitudes
<code>test_reference.py</code>	7	unit	calibration, risk percentile, PSI bins, model guard
<code>test_alerting.py</code>	8	unit	alert budget, rolling threshold, fallback threshold
<code>test_predict.py</code>	15	model	model loads, output shape and range, behaviour on known inputs
<code>test_parity.py</code>	6	model	service = notebooks (\$3)
<code>test_pipeline.py</code>	17	integration	end to end, one log line per order, <code>on_failure</code> , GE on batches
<code>test_monitoring.py</code>	11	unit	JSONL log, PSI, drift snapshot, test-log isolation
<code>test_evaluation.py</code>	9	unit	calendar-day label, join by <code>order_id</code> , metrics
<code>test_artifacts.py</code>	8	unit	version hash, DVC pointers, scikit-learn version guard
<code>test_api.py</code>	17	integration	every route end to end through FastAPI's <code>TestClient</code> , every 422

Module	Tests	Kind	Covers
test_registry.py	9	integration	register (alias, stage, tags), download + md5 check, tampering refused, fallback

Tests write their logs to a temporary directory, never to `logs/predictions.jsonl`, and read the model from `models/`. `test_registry.py` uses a throw-away MLflow store.

Breaking it on purpose — a code bug

To check that the tests catch a realistic refactoring mistake, the Earth radius in the haversine formula was changed from 6,371 km to 6,378 km (the equatorial radius instead of the mean radius). The change was then reverted.

EVIDENCE 14 — one changed constant, caught by the parity tests

```
-EARTH_RADIUS_KM = 6371.0
+EARTH_RADIUS_KM = 6378.0

$ python -m pytest tests/test_parity.py tests/test_preprocessing.py # output filtered to the failures
E   AssertionError: Features differ from the notebook:
E   assert np.float64(9.534612611823832) < 1e-09
E   AssertionError: assert np.float64(0.0002516507729831141) < 1e-12
E   assert 0.7106 == 0.7107
FAILED tests/test_parity.py::TestFeatureParity::test_features_equal_notebook
FAILED tests/test_parity.py::TestFeatureParity::test_sample_order_is_real
FAILED tests/test_parity.py::TestScoreParity::test_scores_equal_model_on_notebook_features
FAILED tests/test_parity.py::TestScoreParity::test_api_path_equals_notebook
FAILED tests/test_parity.py::TestFullTestSplit::test_roc_auc_matches_notebook_6
===== 5 failed, 12 passed in 9.98s =====
```

The bug moved distances by up to 9.5 km but moved ROC-AUC only in the fourth decimal (0.7107 → 0.7106). A metric check alone would not have noticed it. The feature-level parity test did, and in CI that failure stops the pipeline (§9).

9. CI/CD

GitHub Actions runs `.github/workflows/task3-ci.yml` on every push and pull request that touches `mlops-task3/`. It has three jobs, and each one runs only if the one before it passed:

1. **lint**: `ruff check .` and `ruff format --check .`
2. **test** (needs: lint): `pip install -r requirements/dev.txt`, then `pytest tests/ -v`
3. **build** (needs: test): builds the image. On `main` it also pushes `ghcr.io/ziadsuleyman/olist-late-predictor:<sha>` and `:latest`, using the workflow's own `GITHUB_TOKEN`.

So an image is published only after its lint and tests have passed.

A failing test stops the pipeline — the real history

The first CI run of Task 3 failed. On Linux, git had converted the line endings of the JSON artifacts (CRLF → LF), so their md5 no longer matched the DVC pointers, and a test caught it. The build job did not run. The fix was `.gitattributes` (`models/*.json -text`), which stores the artifacts byte for byte; the next run was green.

EVIDENCE 15 — the repository's CI runs, from the GitHub API

```
run 35444206539 1e34c66 failure Task 3: ops scripts, Docker Compose stack, CI workflow, docs
Lint & format check success (ruff check . / ruff format --check .)
Tests failure (step 5: pytest tests/ -v --tb=short)
Docker build skipped <- the image was never built or pushed

run 35444923095 7ddd75f success Task 3: store serving artifacts byte-for-byte so md5s match on Linux

run 35449912777 a5b49cb success Task 3: CI pushes the image to ghcr.io after lint and tests pass
Lint & format check success
Tests success
Docker build & push success (Build / Push to ghcr.io)
```

Before the push: pre-commit

`.pre-commit-config.yaml` runs these hooks before every commit:

- `check-yaml`, `check-json`, `check-toml`
- `end-of-file-fixer`, `trailing-whitespace`
- `no-commit-to-branch main`
- `ruff --fix` and `ruff-format`, at the same ruff version as CI (0.16.7)

On the clean clone: `ruff check .` → All checks passed! ; `ruff format --check .` → 45 files already formatted.

Task 3 went into the repository as small commits, one concern each:

1. structure, config and pinned requirements
2. pipeline modules and the service
3. tests
4. operations and Compose
5. the line-ending fix
6. the registry loading
7. the image push

10. Logging and monitoring

10.1 Logging

Nothing uses `print` in the service. `src/logger.py` writes to the console and to `logs/service.log` (rotating, 10 MB × 5), in the format `[time] [level] [logger] message`. The level comes from `LOG_LEVEL`. In addition, **every order** (single or batch) becomes one line in `logs/predictions.jsonl`. Each line holds the input, the output, the latency and the model version, plus the `order_id` so that the prediction can be scored later.

EVIDENCE 16 — the service log and one line of the prediction log (clean stack)

```
[2026-09-24 16:56:56] [INFO] [olist] Prediction: 1 order(s) late=0 threshold=0.7538 latency=165.1ms
model=f9fcc48ef9ef
[2026-09-24 16:56:56] [INFO] [olist] Prediction: 1 order(s) late=1 threshold=0.7538 latency=68.2ms
model=f9fcc48ef9ef
[2026-09-24 16:56:57] [INFO] [olist] GE validation passed: 70/70 expectations OK
[2026-09-24 16:56:57] [INFO] [olist] Batch prediction: 2 order(s) late=1 threshold=0.7538 latency=783.5ms
model=f9fcc48ef9ef

{"timestamp": "2026-09-24T16:56:56.283768+00:00", "order_id": "8a9be36ffd78382f9ac518945e909636",
 "input_hash": "5398213a11ed", "prediction": "on_time", "probability": 0.0221, "score": 0.3197,
 "risk_percentile": 20.8, "alert_threshold": 0.7538, "model_version": "f9fcc48ef9ef", "latency_ms": 165.14,
 "batch_size": 1, "validation_warnings": null,
 "input": {"order_purchase_timestamp": "2018-06-01 04:19:11", "customer_state": "MG", ... all 24 fields}}
```

10.2 Service metrics — GET /metrics

Metric	Type	Measures
prediction_requests_total{prediction}	counter	Request count, split late / on_time
prediction_latency_seconds	histogram	Latency (p50 / p95 / p99)
prediction_errors_total{error_type}	counter	Errors, validation vs internal ; divided by requests = error rate
prediction_batch_size	histogram	Batch sizes
prediction_validation_warnings_total	counter	Orders served with warnings under flag
prediction_score_psi	gauge	Drift: PSI of recent scores vs. the validation split
prediction_alert_ratio	gauge	Share of recent orders flagged late
prediction_mean_probability	gauge	Mean calibrated probability of recent orders

EVIDENCE 17 — /metrics after the demo requests and the eight bad inputs of §6

```
prediction_requests_total{prediction="on_time"} 2.0
prediction_requests_total{prediction="late"} 2.0
prediction_latency_seconds_count 3.0
prediction_latency_seconds_sum 1.01688
prediction_errors_total{error_type="validation"} 7.0
prediction_batch_size_count 1.0
prediction_alert_ratio 0.5
prediction_mean_probability 0.3528
```

The error counter reads 7: bad inputs 1–7 of §6, schema errors included. Case 8 (the GE batch) was sent after this read.

10.3 Prediction distribution and drift

Drift is measured as the **PSI of the score distribution** against the validation split (warn at ≥ 0.10 , alert at ≥ 0.25). The share of orders flagged late cannot show drift, because the alert budget holds it near 5%. The real late rate cannot be the reference either, because it moved from 7.95% to 3.61% between the splits. The score distribution sums up the inputs, so it is what moves.

To show this on the running system, all **18,603 test-split orders** were sent to the clean stack's `/predict/batch` in purchase order, in batches of 500 (36 s in total). `/drift` was read at the end of each month:

Test month	Orders	PSI (last 1,000 orders)	Status	Flagged late	Mean probability
June 2018	6,096	0.076	ok	6.2%	4.8%
July 2018	6,156	0.556	alert	5.6%	8.5%
August 2018	6,351	0.665	alert	3.2%	7.6%
Whole test split (log)	18,603	0.124	warn	5.4%	7.0%

The whole-split figure comes from `scripts/analyze_predictions.py --all` on the prediction log. The same period is when the delivery promise, the model's most important feature, shrank (median promised days 27 → 19 → 13, June to August) and when ROC-AUC fell from 0.774 (validation) to 0.711 (test).

10.4 Evaluating later, when the real delivery date arrives

Every log line carries its `order_id`, so predictions can be joined to the `orders` table once the orders are delivered. `scripts/evaluate_outcomes.py` does this with Task 2's calendar-day label. Here it was run on the prediction log the clean stack had just written, against the Task 1 database:

EVIDENCE 18 — the prediction log scored against real deliveries

```
$ python scripts/evaluate_outcomes.py --all
=====
PRODUCTION OUTCOMES
=====
{ "n_logged": 18603, "n_with_order_id": 18603, "n_delivered": 18603,
  "late_rate": 0.0361, "roc_auc": 0.7107, "pr_auc": 0.0849,
  "flagged_share": 0.0542, "precision": 0.1021, "recall": 0.1533,
  "mean_probability": 0.0697, "brier": 0.0379 }
Calibration gap (mean probability - real late rate): +3.36%

$ python scripts/analyze_predictions.py --all
Total predictions: 18603 (in 39 requests)
With order_id: 18603 (100.0%) - scorable later
Flagged late: 5.4% (alert budget 5%)
Latency, batch (39 requests): p50 710.3 ms p95 1117.5 ms p99 1148.4 ms
Model versions: f9fcc48ef9ef: 18603 predictions
Score PSI vs validation split: 0.124 -> warn
```

ROC-AUC and PR-AUC here equal Notebook 6's test numbers, this time measured from the service's own log and the database. The calibration gap of +3.36 points has a known cause: the late rate fell from 5.53% on the validation split, where the calibration was fitted, to 3.61% on the test split.

10.5 What to alert on

Signal	Rule	Action
<code>prediction_score_psi</code>	≥ 0.10 warn, ≥ 0.25 alert	Input drift: check the inputs, then recalibrate or retrain
<code>prediction_alert_ratio</code>	far from the 5% budget	The rolling threshold is not tracking the traffic
<code>evaluate_outcomes.py</code> (scheduled)	ROC-AUC falls; calibration gap opens	Recalibrate on recently delivered orders, or retrain
<code>prediction_errors_total{error_type}</code>	rising share of requests	validation : an upstream data change · internal : a service fault

The thresholds are in `config/settings.yaml` (`monitoring.psi_warn`, `psi_alert`, `alerting.budget`). The first three rules are also written in the post-deployment checklist in `docs/DEPLOYMENT.md`.

11. Every config value

All runtime settings are in `config/settings.yaml`, and secrets are in `.env`. `src/config.py` loads both, lets environment variables override the YAML, and stops the start-up on an invalid value.

Key	Value	Why
<code>project.version</code>	<code>1.2.0</code>	Service version reported by <code>/health</code> ; the model version is separate (a hash)
<code>model.source</code>	<code>registry</code>	Load from MLflow; <code>local</code> reads <code>models/</code> (the tests use it)
<code>model.registry_name</code> / <code>_alias</code> / <code>_stage</code>	<code>olist-late-predictor</code> / <code>production</code> / <code>Production</code>	Which registered version to serve; the stage is kept for readers who look for it
<code>model.registry_fallback_to_local</code>	<code>true</code>	Keep serving the same bytes if MLflow is down (\$7.3)
<code>model.*_path</code>	<code>models/...</code>	The four artifacts: no path is written in the code
<code>model.label_definition</code>	<code>calendar-day rule</code>	Task 2's decision: the promise is a date, not an instant
<code>inference.refit</code>	<code>false</code>	Never fit at inference; <code>true</code> is refused
<code>inference.batch_max_size</code>	<code>500</code>	Bounds memory and latency per request (case 7 of \$6)
<code>alerting.budget</code>	<code>0.05</code>	Flag the riskiest 5%: Notebook 6 showed a budget carries across periods, a fixed threshold does not
<code>alerting.window</code> / <code>min_window</code>	<code>1000</code> / <code>200</code>	Recent scores used to place the threshold; below 200, use the validation threshold 0.7538
<code>logging.level</code> / <code>format</code> / <code>file</code>	<code>INFO</code> / <code>[time] [level]</code> / <code>[name] msg</code> / <code>logs/service.log</code>	Level overridable by <code>LOG_LEVEL</code>
<code>logging.max_bytes</code> / <code>backup_count</code>	<code>10 MB</code> / <code>5</code>	Log rotation, so the disk cannot fill
<code>monitoring.prediction_log_file</code>	<code>logs/predictions.jsonl</code>	One line per order, scorable later
<code>monitoring.drift_window</code> / <code>drift_min_window</code>	<code>1000</code> / <code>200</code>	PSI needs enough scores to be stable
<code>monitoring.psi_warn</code> / <code>psi_alert</code>	<code>0.10</code> / <code>0.25</code>	The usual PSI bands
<code>validation.on_failure</code>	<code>reject</code>	A wrong order is refused rather than scored silently; <code>flag</code> is the alternative
<code>validation.ge_on_batch</code>	<code>true</code>	Run the population checks on batches (about 0.9 s each)

Key	Value	Why
<code>validation.allowed_states /</code> <code>allowed_payment_types /</code> <code>numeric_ranges /</code> <code>coordinate_ranges</code>	27 states, 4 types, GE ranges	Taken from the training data; longitude up to -28 because 1,519 real orders in PE and PB lie east of -35
<code>.env : DB_*, MLFLOW_TRACKING_URI ,</code> <code>API_PORT , MODEL_SOURCE</code>	template: <code>.env.example</code>	Secrets and connection strings; the file is gitignored

12. Every tool, and why it is there

Tool	Role in this project
FastAPI + Pydantic	The HTTP service. The schemas reject bad payloads before any code runs, and they generate <code>/docs</code> .
Uvicorn	The server process that runs FastAPI inside the container.
scikit-learn, pandas, numpy	The Task 2 model and transformers, loaded at the exact versions they were pickled with.
Great Expectations	Statistical checks on batches: ranges, sets, missing rates.
DVC	Versions the artifacts; its md5 pointers are the reference the service checks against.
MLflow	Tracks the run (parameters, metrics, artifacts); the registry the API loads from; the artifact store the container can reach.
PostgreSQL	The Olist schema: the source of real outcomes for evaluating predictions later.
Prometheus client	Exposes request count, latency, errors and drift at <code>/metrics</code> .
Python <code>logging</code>	Levels, one format, file and console, with rotation.
pytest (+ httpx)	173 tests with one command; httpx backs FastAPI's TestClient for the route tests.
ruff	Lint and format check, the same version in pre-commit and in CI.
pre-commit	Runs ruff and file checks before each commit, before CI sees it.
Docker / Compose	One image for the service; one command for the four-service stack.
GitHub Actions + ghcr.io	lint → tests → build; publishes the image only after all three pass.